

Teque

Pseudokode

Algorithm 1 Add new number to the back of the list

Input: An integer x

Procedure push_back(x)

```
1:  $|A_{new}| \leftarrow |A| + 1$ 
2: for  $i \leftarrow 0$  to  $|A_{new}| - 1$  do
3:    $A_{new}[i] \leftarrow A[i]$ 
4:  $A_{new}[|A_{new}| - 1] \leftarrow x$ 
5:  $A \leftarrow A_{new}$ 
```

Algorithm 2 Add new number to the front of the list

Input: An integer x

Procedure push_front(x)

```
1:  $|A_{new}| \leftarrow |A| + 1$ 
2:  $A_{new}[0] \leftarrow x$ 
3: for  $i \leftarrow 0$  to  $|A_{new}| - 1$  do
4:    $A_{new}[i + 1] \leftarrow A[i]$ 
5:  $A \leftarrow A_{new}$ 
```

Algorithm 3 Add new number to the middle of the list

Input: An integer x

Procedure push_middle(x)

```
1:  $|A_{new}| \leftarrow |A| + 1$ 
2:  $mid \leftarrow |A_{new}| / 2$ 
3: for  $i \leftarrow 0$  to  $mid$  do
4:    $A_{new}[i] \leftarrow A[i]$ 
5:  $A_{new}[mid] \leftarrow x$ 
6: for  $i \leftarrow mid$  to  $(|A_{new}| - 1)$  do
7:    $A_{new}[i + 1] \leftarrow A[i]$ 
8:  $A \leftarrow A_{new}$ 
```

Algorithm 4 Get number from the list

Input: A integer i that is an index of the list**Output:** An integer that is the value of $A[i]$ **Procedure** $\text{get}(i)$ 1: **print** $A[i]$

Verste-tilfelle kjøreanalyse

Vi kan fordele koden i fem kategorier:

- Input Reading
- List Initialization
- Loop 1
- Loop 2
- Loop 3

Input reading har i verste-tilfelle kompleksiteten $\mathcal{O}(N)$. Dette er på grunn av den går gjennom loopen N ganger.

List initialization har i verste-tilfelle kompleksiteten $\mathcal{O}(1)$. Dette er på grunn av det å lage en variabel har en konstant kompleksitet.

Loop 1, 2 og 3, har i verste-tilfelle hver en kompleksitet av $\mathcal{O}(N)$. Dette er på grunn av den går gjennom hver loopen N ganger.

For å regne den verste-tilfelle kompleksitet for koden, adderer vi disse kompleksitetene til hverandre.

$$\mathcal{O}(N) + \mathcal{O}(1) + \mathcal{O}(N) + \mathcal{O}(N) + \mathcal{O}(N) = \mathcal{O}(N) + \mathcal{O}(N) + \mathcal{O}(N) + \mathcal{O}(N) = 4 \cdot \mathcal{O}(N) = \mathcal{O}(4N)$$

Siden i $\mathcal{O}$ -notasjon vi droper konstantene, så har vi at den verste-tilfelle kompleksitet er $\mathcal{O}(N)$.

Begrensning av N og kompleksiteten i $\mathcal{O}$ -notasjon

Hvis vi vet at N er begrenset, det betyr at N er en konstant, og derfor det input størrelse endrer seg ikke. Dette betyr at kompleksiteten blir ikke avhengig av størrelse til input, og derfor blir $\mathcal{O}(1)$. Dette betyr at kjøretiden er konstant.

Eksperimentér

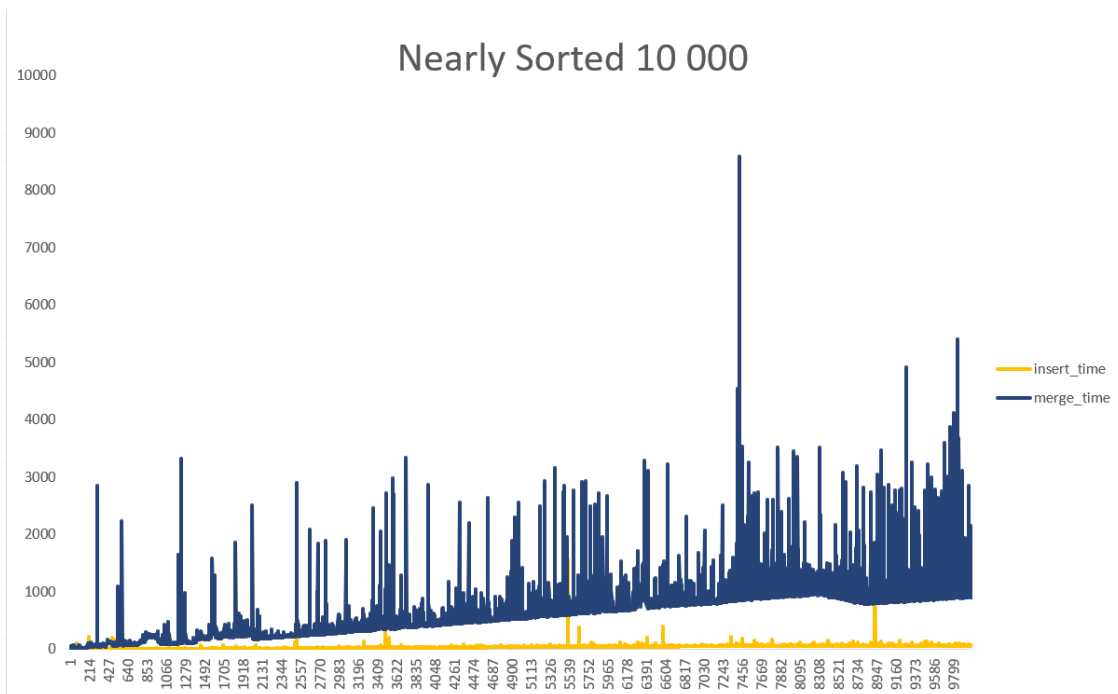
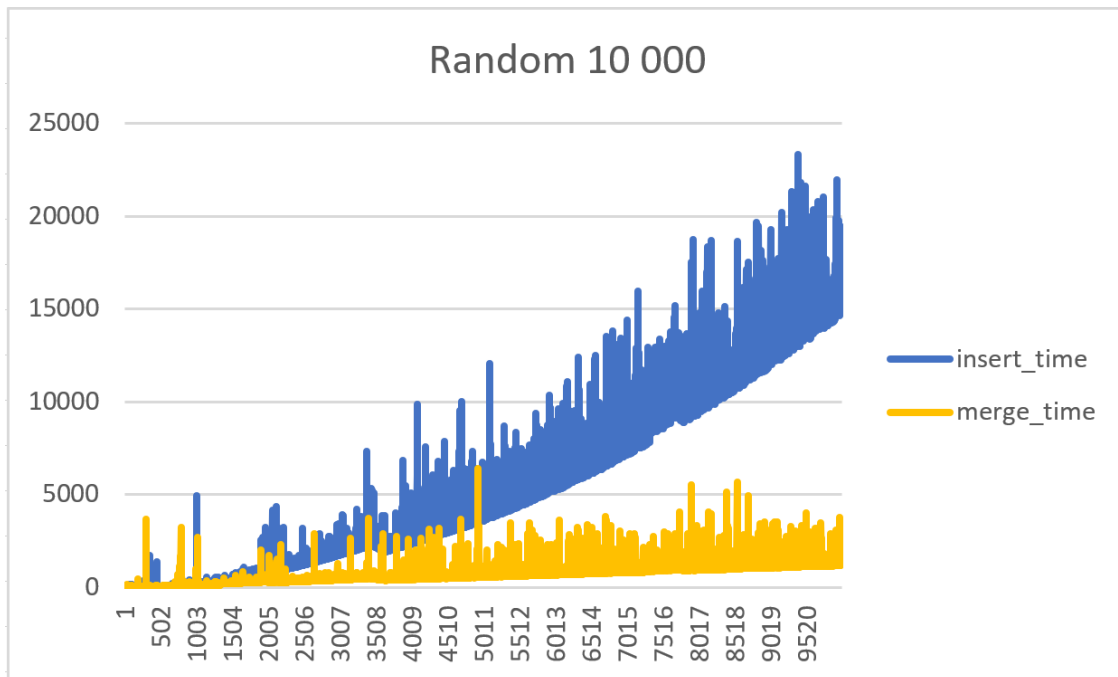
Kjøretid og kjøretidsanalysen

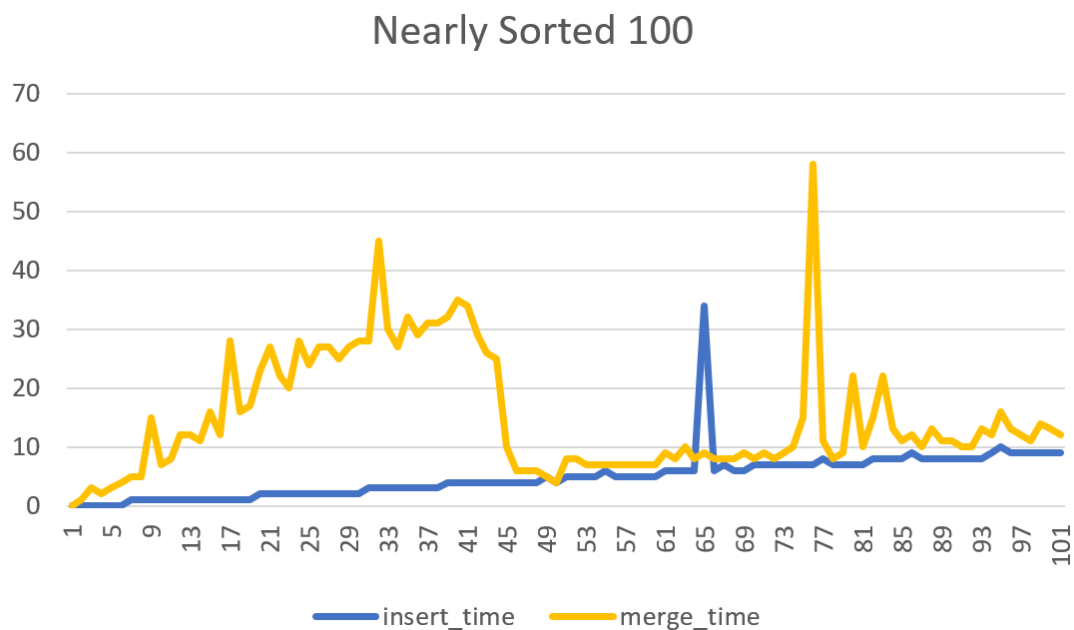
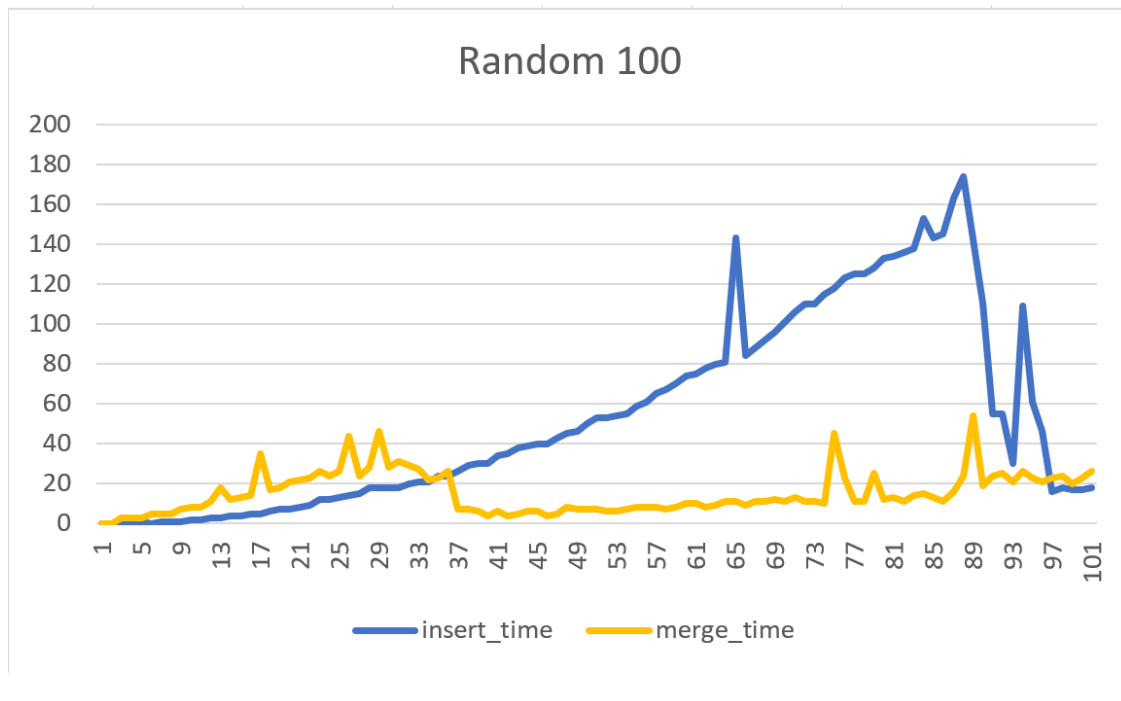
For insertion Sort:

Kjøretiden til insertion sort øker som forventet når størrelsen på datasettet (n) øker. Dette stemmer godt overens med den teoretiske kjøretidsanalysen for insertion Sort, som har en gjennomsnittlig tid på $\mathcal{O}(n^2)$. Når n øker, må algoritmen utføre flere sammenligninger og bytter, noe som resulterer i økt kjøretid. Siden $\mathcal{O}(n^2)$ referer til den verste-tilfelle, så ser vi at dette skjer for random inputfilene.

For Merge Sort:

Kjøretiden til Merge Sort øker også som forventet når n øker. Dette er i tråd med den teoretiske kjøretidsanalysen for Merge Sort, som har en gjennomsnittlig tid på $\mathcal{O}(n \cdot \log(n))$. Selv om kjøretiden øker med økende n , vokser den mye langsommere enn for insertion sort, takket være merge sorts mer effektive natur. Vi ser at kjøretiden til Merge Sort stemmer best med den kjøretidsanalysen når vi ser på random og/eller store n .





Sammenligninger, bytter og kjøretid

Antall sammenligninger og bytter er en viktig indikator på hvor mange operasjoner en sorteringsalgoritme utfører for å sammenligne elementer og bestemme deres rekkefølge. Generelt jo flere sammenligninger og bytter som utføres, desto mer tid tar det. I enkelte steder i grafen ser vi at vi får stor forskjell i kjøretiden selv om det er ikke så stor forskjell i sammenligninger og bytter i forhold til den forrige og neste n .

Utmerking av sorteringsalgoritmene

For Insertion Sort:

Det kom tydelig fram at Insertion Sort var mest effektiv for små verdier av n (stortsett $n < 10$, uavhengig av dataset). Den fungerte best på Nearly_Sorted_10000, gitt at det krevde færre bytter som var forventet. Insertion Sort var mindre effektivt på Random_10000, her krevde det flere bytter og sammenligninger som sterkt påvirket kjøretiden.

For Merge Sort:

Merge Sort var best på store og/eller random dataset (n).

Overraskelser

- Vi forventet at kjøretiden var lengst når n var størst, men dette stemte ikke siden det var flere spikes gjennom kjøretiden.
- Når inputfilen var random_10000, gjorde den insert metoden nesten 25 millioner swaps og 25 millioner sammenligninger, i mens merge gjorde cirka 124 tusen swaps og 254 tusen sammenligninger.
- Når listen var nesten ferdig sortert, var kjøretiden til insert nesten lineær